

CellSafe Application Installation Manual

Unified Guide for Ubuntu 22.04 LTS and SUSE Linux Enterprise Server 15 SP6/SP7

This manual provides instructions for installing the CellSafe Application on two major Linux distributions:

- Ubuntu 22.04 LTS (common for development and cloud deployments)
- SUSE Linux Enterprise Server (SLES) 15 SP6 or SP7 (enterprise-focused, long support cycles; SP7 released mid-2025, fully supported into 2030s)

CellSafe is a PHP-based web application using PostgreSQL (with `pg_cron` extension for scheduled database jobs), Nginx as the web server, and PHP-FPM for processing.

Key Differences Summary

- Package manager: `apt` (Ubuntu) vs `zypper` (SLES)
- PostgreSQL service naming: `postgresql` (Ubuntu) vs `postgresql-<major>` (SLES, e.g., `postgresql-16`)
- Paths: `/etc/postgresql/<version>/main/` (Ubuntu) vs `/var/lib/pgsql/<version>/data/` (SLES)
- PHP socket: `/var/run/php/php8.3-fpm.sock` (Ubuntu) vs `/run/php/php-fpm.sock` (SLES, version-dependent)
- Web root suggestion: `/var/www/cellsafe` (Ubuntu) vs `/srv/www/cellsafe` (SLES convention)

Prerequisites (Both Distributions)

- Fresh server installation with root/sudo access
- Internet access
- Firewall allowing ports 80/443 (HTTP/HTTPS) and optionally 5432 (PostgreSQL, local only recommended)
- At least 2 GB RAM, 20 GB disk
- Replace placeholders: ``yourpassword``, ``example.com``, paths as needed

Step 0: System Update

Ubuntu

```
sudo apt update && sudo apt upgrade -y
sudo apt install curl git unzip -y
```

SLES

```
sudo zypper refresh
sudo zypper update -y
sudo zypper install -y curl git unzip vim
```

Reboot if kernel updated: `sudo reboot`

Step 1: Install PostgreSQL

Ubuntu 24.04 (PostgreSQL 16 default)

```
sudo apt install postgresql postgresql-contrib -y
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

SLES 15 SP6/SP7 (Use PGDG repo for latest; PostgreSQL 16/17 recommended in 2026)

```
sudo zypper install --allow-unsigned-rpm
https://download.postgresql.org/pub/repos/zypp/repopms/SLES-15-x86_64/pgdg-suse-repo-
latest.noarch.rpm
sudo zypper refresh
sudo zypper install -y postgresql16 postgresql16-server postgresql16-contrib # or postgresql17 if
preferred
sudo /usr/pgsql-16/bin/postgresql-16-setup initdb
sudo systemctl start postgresql-16
sudo systemctl enable postgresql-16
```

Verify (both):

```
sudo -u postgres psql -c "\conninfo"
```

Create Database/User/Schema (Same on Both)

```
sudo -u postgres psql
```

```
CREATE USER cell WITH PASSWORD 'cell2025';
CREATE DATABASE Cell OWNER cell;
GRANT ALL PRIVILEGES ON DATABASE Cell TO cell;
```

```
CREATE SCHEMA IF NOT EXISTS cells AUTHORIZATION cell;  
\q
```

Load Schema Objects

CellSafe provides schema definitions as SQL files: schema.sql

Option 1: Redirect input (simple and widely used)

```
psql -U postgres -d Cell < /path/to/your/schema.sql
```

Option 2: Using -f flag (shows line numbers on errors)

```
psql -U postgres -d Cell -f /path/to/your/schema.sql
```

Step 2: Install pg_cron (Database Cron Add-in)

Ubuntu

```
sudo apt install postgresql-server-dev-all build-essential libpq-dev -y  
git clone https://github.com/citusdata/pg_cron.git  
cd pg_cron  
make && sudo make install
```

SLES

```
sudo zypper install -y gcc make postgresql16-devel # adjust version  
git clone https://github.com/citusdata/pg_cron.git  
cd pg_cron  
make && sudo make install
```

Common Configuration

Edit config file:

- Ubuntu: `sudo nano /etc/postgresql/16/main/postgresql.conf` (adjust version)
- SLES: `sudo nano /var/lib/pgsql/16/data/postgresql.conf` (adjust version)

Add:

```
shared_preload_libraries = 'pg_cron'  
cron.database_name = 'Cell'
```

Restart:

- Ubuntu: `sudo systemctl restart postgresql`
- SLES: `sudo systemctl restart postgresql-16`

Enable extension:

```
sudo -u postgres psql -d Cell -c "CREATE EXTENSION pg_cron;"
```

Test:

```
sudo -u postgres psql -d Cell -c "SELECT cron.schedule('test', '*/5 * * * *', 'VACUUM');" 
```

Step 3: Install Nginx

Ubuntu

```
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
sudo ufw allow 'Nginx Full' # if using UFW
```

SLES

```
sudo SUSEConnect -p sle-module-web-scripting/15.5/x86_64 # adjust SP if needed
sudo zypper install -y nginx
sudo systemctl start nginx
sudo systemctl enable nginx
sudo firewall-cmd --permanent --add-service=http --add-service=https
sudo firewall-cmd --reload
```

Verify: Browse server IP → Nginx welcome page.

Step 4: Install PHP-FPM

Ubuntu (PHP 8.3 common; adjust if needed)

```
sudo apt install software-properties-common -y
sudo add-apt-repository ppa:ondrej/php -y
sudo apt update
sudo apt install php8.3 php8.3-fpm php8.3-pgsql php8.3-curl php8.3-mbstring php8.3-xml php8.3-gd
php8.3-zip -y
sudo systemctl start php8.3-fpm
sudo systemctl enable php8.3-fpm
```
```

## SLES

```
sudo zypper install -y php8 php8-fpm php8-pgsql php8-curl php8-mbstring php8-xml php8-gd php8-
zip php8-dom php8-json
sudo systemctl start php-fpm
sudo systemctl enable php-fpm
```

**Verify:** `php -v`

## Step 5: Deploy CellSafe Application

### Create directory:

Suggest to create user cellsafe and group cellsafe, and login as cellsafe

- Ubuntu: `sudo mkdir -p /var/www/cellsafe && sudo chown -R $USER:$USER /var/www/cellsafe`
- SLES: `sudo mkdir -p /srv/www/cellsafe && sudo chown -R $USER:www /srv/www/cellsafe`

### Permissions:

```
sudo chown -R www-data:www-data /var/www/cellsafe # Ubuntu
or
sudo chown -R wwwrun:www /srv/www/cellsafe # SLES
```

## Install Application

Cellsafes provides two zip files, server.zip and cellsafe.zip, unzip both files in above create directory. There will be two folders under the installation directory: server and cellsafe

## Step 6: Configure Nginx

## Create site config:

**Please acquire the SSL certificate for the server**

Ubuntu: /etc/nginx/sites-available/cellsafe

SLES: /etc/nginx/vhosts.d/cellsafe.conf

Example content (adjust root and PHP socket):

```
server {
 listen 80 default_server;
 listen [::]:80 default_server;

 # SSL configuration
 #
 listen 443 ssl default_server;
 listen [::]:443 ssl default_server;
 #
 # Note: You should disable gzip for SSL traffic.
 # See: https://bugs.debian.org/773332
 #
 # Read up on ssl_ciphers to ensure a secure configuration.
 # See: https://bugs.debian.org/765782
 #
 # Self signed certs generated by the ssl-cert package
 # Don't use them in a production server!
 #
 #include snippets/snakeoil.conf;
 ssl_certificate /etc/ssl/certs/ssl-cert.pem;
 ssl_certificate_key /etc/ssl/private/ssl-cert.key;

 root /var/www/cellsafe;

 # Add index.php to the list if you are using PHP
 index index.php index.html index.htm index.nginx-debian.html;

 server_name _;

 location / {
 # First attempt to serve request as file, then
 # as directory, then fall back to displaying a 404.
 try_files $uri $uri/ =404;
 }

 # pass PHP scripts to FastCGI server
 #
 location ~ \.php$ {
 include snippets/fastcgi-php.conf;

 # With php-fpm (or other unix sockets):
 fastcgi_pass unix:/run/php/php-fpm.sock;
```

```

 # With php-cgi (or other tcp sockets):
 #fastcgi_pass 127.0.0.1:9000;
 }

 # deny access to .htaccess files, if Apache's document root
 # concurs with nginx's one
 #
 location /cellsafe/ {
 # Use alias to point to the correct file path
 index index.html;

 # Essential for Angular routing: rewrite all requests in this location
 # to the index.html file, passing the original path as an argument.
 try_files $uri $uri/ /cellsafe/index.html ;
 }
 location ~ /\.ht {
 deny all;
 }
}

```

## Enable & reload:

- Ubuntu: `sudo ln -s /etc/nginx/sites-available/cellsafe /etc/nginx/sites-enabled/`
- Both: `sudo nginx -t && sudo systemctl reload nginx`

## Step 6: Testing & Security

- Access: `https://yourhost.your-site.domain`
- Logs: Nginx ``/var/log/nginx/error.log``; PostgreSQL version-specific logs
- Secure PostgreSQL: Edit ``pg_hba.conf`` for local/trusted access only
- Backups: `pg_dump` or tools like `pgBackRest`

## Appendix: Sample Data

Four csv files are provided for the initial data catalog for Genes and Cell lines, you can load all data into database by using `pgadmin`

1. `nbbi_gene_list.csv` (download from NLM and edit for the database)
2. `gene_synonyms.csv`
3. `cell_data.csv` (Download from the Human Protein Atlas project) – optional initial set of cell lines.
4. `cellosaurus_synonyms.csv` (cell aliases from Cellosaurus)